# DB13

**Relational database design algorithm**

# Relation Decomposition

> **Decompistion을 이용한 Relational database design**

- ER Modeling 등을 이용할 수 있지만 이 방법이 **공식적인 (Formal) 방법**이다.

1. **Universal relation schema $R$로부터 시작한다.**
   - Database의 모든 attribute를 포함하는 하나의 Relation schema
   - 모든 Attribute의 이름은 **Unique** 해야 한다.
2. **Universal relation shcema $R$를 여러 Relation schema $D$로 분해한다.**
   - **Universal relation schema에 존재하는 Attribute는 최소한 하나 이상의 Relation schema $D$에 포함**되어야 한다.
     - $D$의 Attribute는 $R$에 존재하는 Attribute이어야 한다.
     - 여러번 나타나는 Attribute는 JOIN을 위한 Attribute이다.
   - **각 Relation은 BCNF 또는 3NF**이어야 한다.
     - BCNF로 만들지 못하는 경우에는 3NF로 한다.
   - $D$: Decompostion of $R$

---

# Insufficienct of Normal Forms

> **Normal form이 보장하지 못 하는 것**

**Attribute가 2개인 Table은 항상 BCNF**이다.

- $A$가 PK일 때, A → B / B → A / A → A 모두 조건을 만족한다.

**Universal relation schema를 정해진 알고리즘을 따르지 않고, Attribute가 2개인 Relation만 존재하도록 나누면 BCNF를 만족**한다.

- 하지만 이 경우, JOIN의 경우의 수가 너무 많아져 **Spurious tuple이 발생**할 수 있다.

따라서 **Good relational database**를 위해선 Normal form과 더불어 아래 두 조건을 만족해야 한다.

1. **Dependency preservation property**
2. **Lossless join property**

---

# Solution

> **Normal form만으로 Lossless Join Property와 Dependency Preserve Property가 보장되지 않을 때, Lossless Join Property와 Dependency Preserve Property를 보장할 수 있도록 하는 방법**

## Dependency preservation property

> **Relation Decompistion의 결과로 생성된 각 Functional dependency들의 합집합이 원래의 Functional Dependency와 동일한 경우**

- $F$: Relation schema $R$의 정의된 FD의 집합
- Relation schema $R$의 Decomposition을 $D_i$라고 하면 $D_i$의 FD인 $F_i$의 합집합이 $F$와 동일해야 한다.
    - **Decomposion $D$에 걸리는 강력한 제약**이다.

## 이론적(formal) 정의

$F^+$: $F$로부터 유도할 수 있는 FD의 집합

$\pi_{R_i}(F)$: $F^+$에 포함된 FD $X \rightarrow Y$ 중, $X \cup Y$가 $R_i$에 포함된 FD의 집합

- Projection of F on $R_i$

**Decomposition $D$가 Dependency-preserving한 경우**

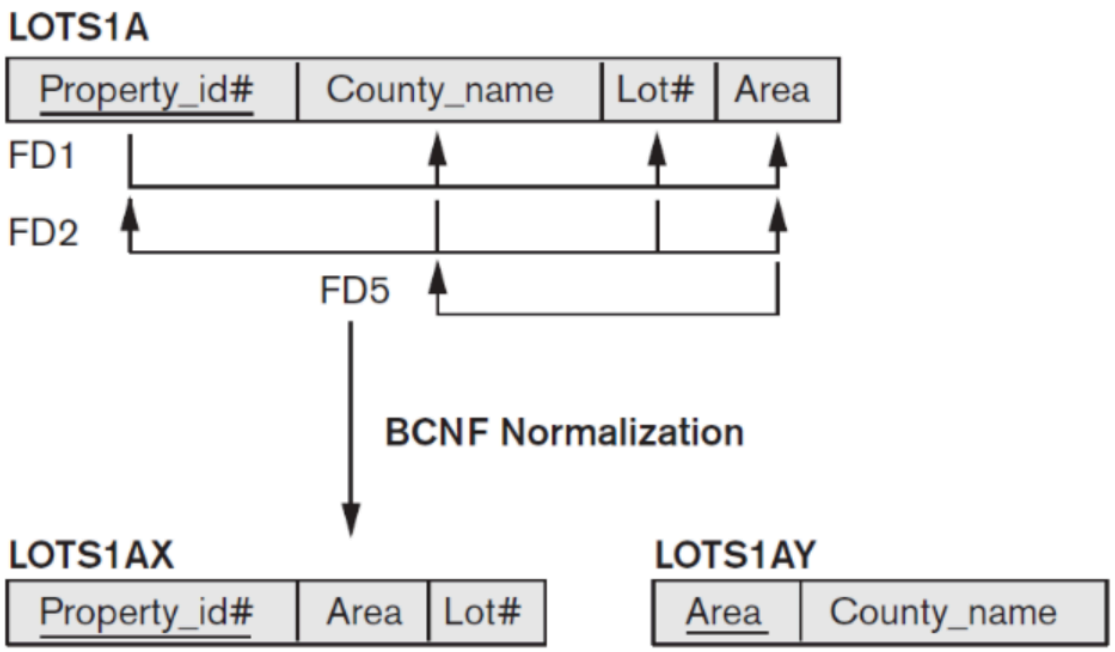
- $F^+ = (\sum_i \pi_{R_i}(F))^+$ 인 경우

## Decomposition $D$가 Dependency-preserving하지 않다면?

- **FD를 구성하는 Attribute들이 다른 Relation으로 분해**되었다는 의미
- BCNF를 사용하는 경우 발생할 수 있다.

**Dependency-preserving하지 않은 경우**에는 원래의 **FD가 존재하는지 확인하기 위해선 JOIN**을 해야 한다.

- JOIN은 시간 소요가 크기 때문에 **비효율적이고 실용적이지 않다.**




- FD2를 검사하기 위해선 JOIN을 해야 한다.

## Relational Decompistion into 3NF with Dependency Preservation

Minimal cover $G$ of $F$: $G^+ = F^+$가 되도록 하는 FD 집합

- G에서 어떤 것 하나라도 제거했을 때 F와 equivalent하지 않게 되는 경우
  - FD를 제거하거나 $X \rightarrow YZ$에서 $X \rightarrow Y$가 되는 경우를 포함한다.

### Minimal cover $G$ of $F$를 찾는 방법

1. **모든 FD를 Canonical form으로 만든다.**
   - Canonical form : RIght-hand side에 하나의 Attribute만 남은 형태
   - $X \rightarrow YZ$ = $X \rightarrow Y, X \rightarrow Z$

2. **각 FD에 대해 Left-hand side의 Redundant attribute가 있다면 제거한다.**
   - $XY \rightarrow Z$에서 $X \rightarrow Z$가 가능하다면 $X \rightarrow Z$로 수정한다.

    - 다른 FD를 이용하여 판별한다.

3. **Redundant한 Fucntional dependence를 제거한다.**
   - $F$에서 다른 FD를 통해 $X \rightarrow Y$를 유도할 수 있다면 자체가 불필요하다면 제거한다.
     - 다른 FD를 이용하여 판별한다.

**알고리즘**

1. **$F$의 Minimal cover $G$를 찾는다.**

2. **$G$의 FD의 Left-hand side인 각 $X$에 대해 아래 동작을 반복한다.**
   - 동일한 **left-hand side** $X$를 가지는 모든 FD들의 right-hand side들을 모아 $R_i = X \cup Y_1 \cup Y_2 \cup \cdots$ 형태의 relation schema를 생성한다
   - 새로 생성된 Relation의 **Key**는 $X$이다.

3. **어떤 Relation에도 들어가지 못한 Attribute는 하나의 Relation schema에 전부 넣는다.**
   - **Attribute preservation property**를 만족하도록 한다.
     - 분해된 모든 relation schema들의 attribute 집합의 합은 원래 relation schema의 attribute 집합과 같아야 한다.

# Lossless Join Property

**Non-additive join property**라고도 불린다.

**Decomposition된 Relation에 JOIN을 적용했을 때, Spurious tuple이 생성되지 않는 것을 보장**한다.

## 이론적 (Formal) 정의

> **원래 Table에 대해 분해하였을 때 얻은 Table instance $\pi_{R_i}(r)$을 각 Table에 대해 Projection하고 JOIN하면 원래의 r이 나온다.**

$$*(\pi_{R_1}(r), \pi_{R_2}(r), ..., \pi_{R_n}(r)) = r$$

- $*$: Natural JOIN, JOIN
- $\pi_{R_i}(r)$: 원래의 Relation instance $r$에서 분해된 Relation $R_i$에 포함된 Attribute에 대응되는 Instance

## Relational Decompistion into BCNF with Lossless Join Property

### 알고리즘

1. $D = R$
2. **$D$에 BNCF가 아닌 Relation $Q$가 존재할 때까지 아래 동작을 반복한다.**
    a. $D$에서 **BNCF가 아닌 하나의 Relation** $Q$를 선택한다.
    b. $Q$에서 **BCNF를 만족하지 않는 하나의 FD ($X \rightarrow Y$)**를 선택한다.
    c. $Q$를 두 개의 Relation $(Q - Y)$와 $(X \cup Y)$로 분해하여 $D$에 $Q$ 대신 두 Relation을 적용한다.
        - $(X \cup Y)$의 Key는 $X$이다.
        - 결국 **$X$가 $(Q - Y)$에선 FK, $(X \cup Y)$에선 PK로 되어 Lossless Join Property를 만족**한다.

### 알고리즘의 두 가지 특성

1. $D = R_1, R_2$ **of** $R$로 Decompose되었다고 했을 때, $F$가 $R$의 FD 집합인 경우, **아래 두 조건 중 하나라도 만족하는 것은 Lossless Join Property를 만족하기 위한 필요충분조건이다.**
   - $(R_1 \cap R_2) \rightarrow (R_1 - R_2) \in F^+$
   - $(R_1 \cap R_2) \rightarrow (R_2 - R_1) \in F^+$

   위 알고리즘에서 Decompose 되는 형태를 살펴보자.

   - $R_1$: $Q - Y$
   - $R_2$: $X \cup Y$
   - $R_1 \cap R_2 = X \rightarrow R_2 - R_1 = Y$이므로 $X \rightarrow Y \in F^+$이다.
     - 따라서 위 조건을 만족한다.

   **key and foreign-key pair**

   - 공통 속성이 한 Relation에서는 Key, 다른 Relation에서는 Foreign key로 사용된다.

2. Relation schema $R$에 대한 FD 집합을 $F$라 하자.
   a. $R$을 $D = R_1, R_2, \ldots, R_m$으로 분해했을 때, $D$가 $F$에 대해 **Lossless Join Property**을 만족하고,
   b. $R_i$를 다시 $D_i = Q_1, Q_2, \ldots, Q_k$로 분해했을 때, $D_i$가 $R_i$에 Projection된 FD 집합에 대해 **Lossless Join Property를** 만족한다면
   c. 원래 Relation schema $R$을 $D_2 = R_1, \ldots, R_{i-1}, Q_1, \ldots, Q_k, R_{i+1}, \ldots, R_m$으로 분해한 결과 또한 $F$에 대해 **Lossless Join Property를** 만족한다.

   즉, **Lossless Join Property를 만족하는 Decomposition을 다시 Lossless Join Property를 만족하도록 Decomposition해도 최종 결과는 Lossless Join Property를 만족한다.**

위 알고리즘에서 **Lossless Join Property를 만족하도록 반복하여 분해하여 최종적으로 Lossless Join Property를 만족하는 Decomposition을 얻을 수 있는 이유**이다.

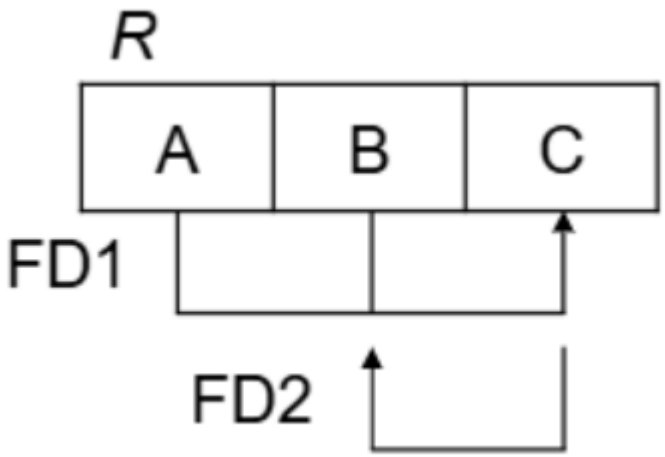
# Relation Decomposition with Dependency Preserve + Lossless Join

## BCNF 이용

**BCNF + Dependency-preserving, Lossless join을 모두 만족시키도록 하는 방법은 없다.**

**Example**

아래 상황에서 BCNF를 위반하는 FD2가 존재할 때, **알고리즘에 따라 이를 두 개의 Relation으로 나누게 되면 FD1이 Preserve되지 않는다.**




## 3NF 이용

**Relational decomposition into 3NF with dependency preservation and lossless join property이 존재한다.**

**알고리즘**

1. **$F$의 Minimal cover $G$를 찾는다.**

2. **$G$의 FD의 Left-hand side인 각 $X$에 대해 아래 동작을 반복한다.**
   - $G$의 FD에서 $X \to A_1, X \to A-2, \ldots, X \to A_k$일 때 $X \cup A_1, A_2, \ldots, A_k$ 를 Attribute로 갖는 Relation $D$를 생성한다.

3. **$R$의 Candidate Key를 어떤 $D$도 온전히 포함하지 않을 때, 하나의 Relation schema를 생성하여 해당 Candidate Key를 포함하도록 한다.**
   - 한 Table에 Candidate key가 온전히 담겨있다면, 이 Relation을 기준으로 항상 올바르게 JOIN할 수 있다.
   - 여러 Candidate key가 있다면 하나를 골라서 추가하면 된다.

**원래 Table $R$의 Candidate key 하나를 찾는 방법**

1. $K := R$
2. **$K$에 존재하는 하나의 Attribute $A$에 대해 아래 동작을 반복한다.**
   a. $R$의$RF$에 대해 $(K-A)^+$를 계산한다.
      - $(K-A)^+$는 $(K-A)$의 Attribute와 FD를 이용하여 계산하여 얻을 수 있는 Attribute의 집합이다.
   b. 만약 $(K-A)^+$가 $R$의 모든 Attribute를 포함한다면 $K := K - A$로 설정한다.

**Relation의 Candidate key 중 하나**를 얻을 수 있다.

# Summary

**BCNF + Lossless join을 사용할 지, 3NF + Lossless join + Dependency preserving을 사용할 지 사이에 딜레마**가 있다.

- BCNF는 3NF에 비해 $X \rightarrow Y$에서 $X$가 Superkey로 한정되어 있기 때문에 Redundant가 줄어 Update anomaly가 적다.
- 그러나 3NF를 사용하는 경우 Dependency preserving을 통해 JOIN 없이도 FD가 보존되는지 확인할 수 있다.

**실제로는 3NF를 만족하는 거의 대부분의 Relation은 BCNF도 만족한다.**